

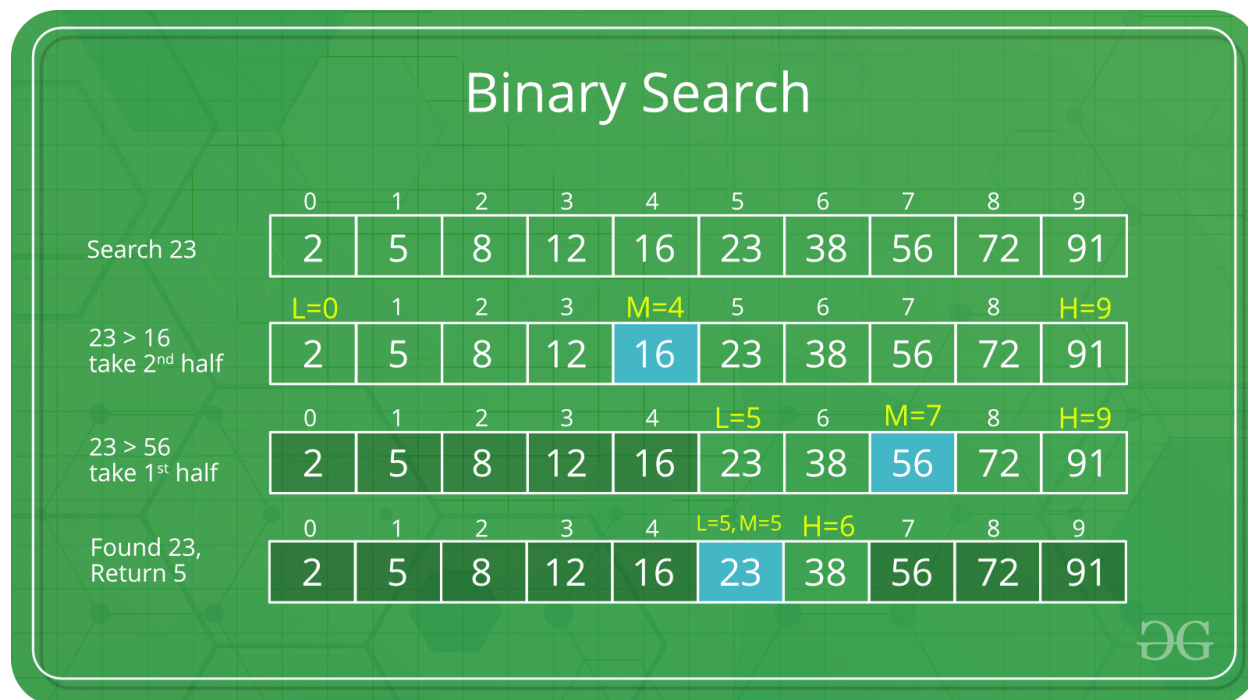
What Is Binary Search?

Binary Search Algorithm Diagram Caption: Step-by-step visualization of binary search narrowing down to a target element.

- A **divide-and-conquer** search algorithm that locates a target value in a **sorted** collection by repeatedly halving the search space.
- Compares the target to the **middle element**; discards the half that cannot contain the target.
- Sourced from Wikipedia: requires only three pointers (low, high, mid), keeping **space usage minimal** regardless of array size.
- Far more efficient than linear search for large datasets — $O(\log n)$ vs $O(n)$.

Example: Searching for 7 in [2, 3, 5, 7, 8, 10, 12] — mid is 7 (index 3), found on the **first comparison**.

Preconditions & Core Mechanics



Caption: Binary search requires a fully sorted array before the first comparison.

- **Precondition:** The array **must be sorted** (ascending or descending); results are undefined otherwise.
- Three pointers maintained each step: $\text{low} = 0$, $\text{high} = n-1$, $\text{mid} = (\text{low} + \text{high}) / 2$.
- **Three outcomes** each iteration:
 - $\text{arr}[\text{mid}] == \text{target} \rightarrow$ found, return index.
 - $\text{arr}[\text{mid}] < \text{target} \rightarrow$ move $\text{low} = \text{mid} + 1$ (search right half).
 - $\text{arr}[\text{mid}] > \text{target} \rightarrow$ move $\text{high} = \text{mid} - 1$ (search left half).
- Loop terminates when $\text{low} > \text{high}$ (target absent $\rightarrow$ return -1).

Example: Target 10 in [1, 3, 5, 7, 10, 14, 18]: mid=7(too small) $\rightarrow$ mid=14(too big) $\rightarrow$ mid=10 (3 steps).

Iterative vs. Recursive Implementation

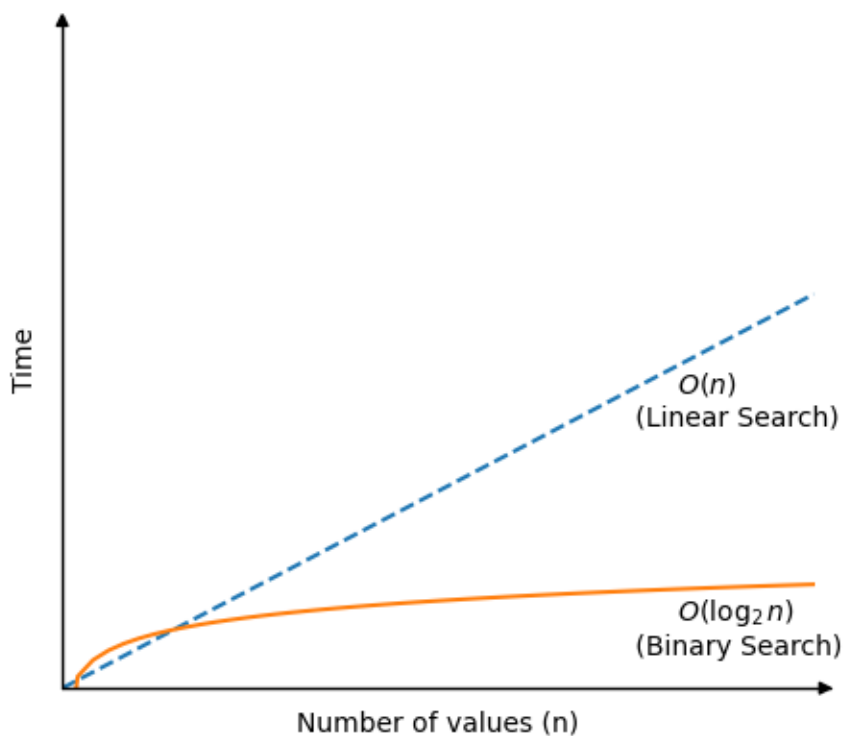
Binary Search Flowchart Caption: Flowchart comparing the decision logic used in binary search iterations.

- **Iterative** approach uses a `while (low <= high)` loop — preferred for its **$O(1)$ space** usage.
- **Recursive** approach calls itself with updated bounds — elegant but adds **$O(\log n)$ call-stack frames**.
- Both yield identical $O(\log n)$ **time complexity** per GeeksforGeeks and TechieDelight sources.

Example (Python — Iterative):

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: low = mid + 1
        else: high = mid - 1
    return -1
```

Time & Space Complexity



Caption: $O(\log n)$

grows extremely slowly compared to $O(n)$, making binary search highly scalable.

- **Best case:** $O(1)$ — target is the first mid element checked.
- **Average & Worst case:** **$O(\log n)$** — array is halved each step; 1,000,000 elements need 20 comparisons.
- **Space complexity:** $O(1)$ iterative / $O(\log n)$ recursive (call stack).

- Example:** An array of 1 billion elements requires at most $\log(10^9) = 30$ comparisons.

Integer Overflow in binary search

1 3 3 4 5 8 30 14 15 83 21

2147483647 → -414%% / ~-%!%

Error: Wrapping!

33278 3343 6677 2333 15

~~mid = (left + right) / 2~~

✓ mid = left + (right - left) / 2

- **Integer overflow:** `mid = (low + high) / 2` can overflow in languages like C/Java; use `mid = low + (high - low) / 2` instead.
- **Off-by-one errors:** Incorrectly using `<` instead of `<=` in the loop condition causes missed elements.
- **Unsorted input:** Binary search silently produces wrong answers on unsorted arrays.
- **Duplicate elements:** Standard binary search may return any matching index; use modified versions to find first/last occurrence.
- **Empty arrays:** Always guard with an initial `if len(arr) == 0` check.

Real-World Applications

- **Standard libraries:** Python's `bisect`, Java's `Arrays.binarySearch()`, C++'s `std::binary_search` all use it.
- **Database indexing:** B-trees extend binary search logic for fast record lookup.
- **Git bisect:** Finds the commit that introduced a bug by binary-searching commit history.

- **Numeric methods:** Used to find square roots, solve monotonic equations, and in “binary search on the answer” competitive programming technique.
- **Networking:** IP routing tables use binary search for longest-prefix matching.

Example: `bisect.bisect_left([1,3,5,7,9], 6)` returns index 3 — the insertion point — in $O(\log n)$ time.